Git: cheatsheet et conventions

Git
- Principes clés et branches
- Les avantages de ce workflow
- 🛠️ Exemple
  - ✨Nouvelle feature
  - ⬇️ Mettre sa branche à jour (rebase / pull)
  - ⚠️ Si un conflit apparaît
  - 📝 Ouvrir une pull request
- 🎁 Commandes avancées
  - J'ai mal résolu des conflits, et veux arrêter le rebase
  - Je veux "revenir dans le passé"
  - J'ai détruit une branche avec un push —force et veux revenir en arrière

Git

Pour collaborer, nous utilisons git avec le workflow **fast-forward-only.** Il permet de garder l'historique clair et minimal.

Principes clés et branches

- `master` : branche de production. On n'y touche pas
- `dev` : branche principale
- une nouvelle feature = une nouvelle branche
- Avant de merger, la branche doit être à jour (via un rebase sur `dev` **).**
- Le merge vers `dev` se fait via une pull request

Les avantages de ce workflow

- **Debug simplifié :** `git bisect` devient ultra efficace

- **PRs plus propres** : on voit ce que la feature apporte réellement
- L'historique de `dev` reste linéaire et minimal. Pas de noeuds et de branches imbriquées

---

🛠️ Exemple

✨Nouvelle feature

On part de `dev` à jour :

```
dev: A -- B -- C
```

Et on crée sa branche :

```
dev:    A -- B -- C
                \
ma-super-feature:    D -- E
```

▼ commandes

```
git pull
git checkout -b "ma-super-feature"
```

⬇️ Mettre sa branche à jour (rebase / pull)

CAS 1: Si `dev` avance pendant que tu travailles :

```
dev:    A -- B -- C -- F -- G
                \
ma-super-feature:    D -- E
```

Alors tu **rebases la feature sur dev** pour réappliquer tes commits par-dessus l'historique le plus récent :

▼ commandes

```
# Option 1
git checkout dev
```

```
git pull # Met à jour le local et le distant
git checkout -
git rebase dev
git push --force-with-lease

# Option 2
git fetch --all # Met à jour le distant
git rebase origin/dev # Rebase sur le distant
git push --force-with-lease
```

```
dev:     A -- B -- C -- F -- G
                        \
ma-super-feature:              D -- E
```

CAS 2: Quelqu'un d'autre a push des commits sur ta branche

```
ma-super-feature (remote):     A -- B -- C -- D

ma-super-feature:    A -- B -- C -- E
```

Alors ne clique **surtout pas** sur le bouton "Sync" de VSCode.

Pour te mettre à jour, pull avec le flag `--rebase` pour que tes changements soient les derniers appliqués.

▼ commandes

```
git pull --rebase
```

⚠️ Si un conflit apparaît

Tu le résous localement pendant le rebase.

Une fois terminé, tu continues ton rebase et push force (`git push --force-with-lease`).

▼ commandes

```
...
git rebase origin/dev
```

```
# conflits : je les résous dans mon IDE, puis je add les fichiers
git rebase --continue # ou cliquer sur "continue" dans ton IDE
# On continue jusqu'à être passé sur tous les commits / conflits
git push --force-with-lease # finaliser le rebase
```

📝 Ouvrir une pull request

Pour ouvrir une PR, aller sur le github et cliquer sur “pull request”. Si on vient de push dessus, on aura directement un bouton proposé. Attention à bien target la branche `dev`

🎁 Commandes avancées

▼ J’ai mal résolu des conflits, et veux arrêter le rebase

```
git rebase --abort
```

▼ Je veux “revenir dans le passé”

Pour diverses raisons on peut vouloir revenir sur un commit passé. La commande pour est `git reset` .

```
git reset (hash du commit)
```

ex: j’ai les commits A - B - C - D. Je veux revenir au commit B

`git reset (hash de B)` : tous les changements apportés par les commits C et D sont conservés dans le local (comme si on avait modifié les fichiers sans add ni commit le changement)

`git reset (hash de B) --hard` : les changements apportés par les commits C et D sont écrasés

▼ J’ai détruit une branche avec un push —force et veux revenir en arrière

“tant que j’ai commit, rien n’est perdu”

Toutes les actions de chaque `ref` (la plus commune étant ta `HEAD` ="ce que tu fais”) sont conservées en local pendant 30 jours dans le `reflog` , sous forme de commits qui ont chacun leur propre hash. On peut donc revenir en

arrière sur une action que l'on a faite, et reset sur ces commits. Pour l'afficher :

```
git reflog
```

Exemple :

```
HEAD -> parametrage, origin/parametrage) HEAD@{8}: rebase (finish): returning to refs/heads/parametrage
f214b69 (HEAD -> parametrage, origin/parametrage) HEAD@{9}: rebase (pick): Ajustement multiple sur bloc 40
d32906a HEAD@{10}: rebase (pick): oubli fichier migration variablebulletin
79dcab5 HEAD@{11}: rebase (pick): correction bug merge
3d69dda HEAD@{12}: rebase (pick): ajout regle particulière pour test régularisation lignes de bulletin avec taux pas à jour
e023bca HEAD@{13}: rebase (start): checkout e023bca
dbdc3b6 HEAD@{14}: pull --tags origin parametrage: Fast-forward
956641b (dev) HEAD@{15}: reset: moving to 956641b890ba471305f11df6b18a1151caf8a99e
```

ici on voit que j'ai fait un rebase depuis la branche paramétrage. Si je veux "annuler ce rebase", je peux faire :

```
git reset dbdc3b6 --hard # Reset sur le commit de la dernière action ava
nt le rebase
```

pour revenir dans le temps avant le rebase. L'action de `git reset` est également enregistrée dans le reflog, on pourra donc revenir en arrière... et ainsi de suite.

Extras

https://www.git-tower.com/learn/git/faq/git-fast-forward

https://medium.com/@sanjaydh006/mastering-rebase-and-ff-only-for-clean-git-histories-b1e48c864613

https://apps.theodo.com/en/article/how-to-become-a-master-of-git-amends-on-any-past-commit